RESEARCH REPORT · CHESS-NN-PLAYGROUND

Architecture Scout

A 234-model survey of bespoke chess-evaluation architectures, with apples-to-apples comparisons across difficulty, phase, eval bucket, and tactic motif.

AUTHOR	Lennart Axel Conrad (Student ID: 2025080264)
AFFILIATION	Zijing College, Tsinghua University
SUPERVISOR	Prof. Jungong Han (Department of Automation, Tsinghua University)

SCOUT DATE	2026-05-09 to 2026-05-10
TASK	puzzle_binary (single positive logit, BCE loss)
DATASET	CRTK-tagged 3-class split (~173k train / ~21k val / ~21k test, zero FEN overlap)
HARDWARE	RTX 3070 (8 GiB) — single GPU, single seed (42), base scale
BUDGET	12 epochs max, patience 3, 60-min wall per task, CUDA only
GENERATED	2026-05-11
REPOSITORY	github.com/LenniAConrad/chess-nn-playground

What this work is

We compare bespoke chess-evaluation architectures on a single small-scale binary task (*puzzle vs non-puzzle*) to identify **which structural priors a chess engine should actually use**. "Best" is judged on two axes simultaneously:

- **Sample efficiency** — test PR AUC at a fixed training budget. Drives Elo per training position.
- **Inference speed** — network evaluations per second on a fixed GPU. Drives Elo per second of search. For an MCTS engine running thousands of nodes per move, a 2× speed edge is typically worth more than a 30-Elo accuracy edge.

The scout (this report) uses puzzle_binary as a small-scale proxy; the engine-relevant outcome is the i243 proposal (§12) that composes the surviving priors for engine-grade training.

Executive summary

234 bespoke chess architectures were trained once each at small scale on a puzzle-detection task. ***175*** *produced usable results; 50 crashed on code bugs (21%); 9 hit the 60-minute training wall (4%).*

234	175	180	0.8755
ARCHITECTURES TRAINED	COMPLETED RUNS	IN LEADERBOARD	TOP TEST PR AUC

Headline finding

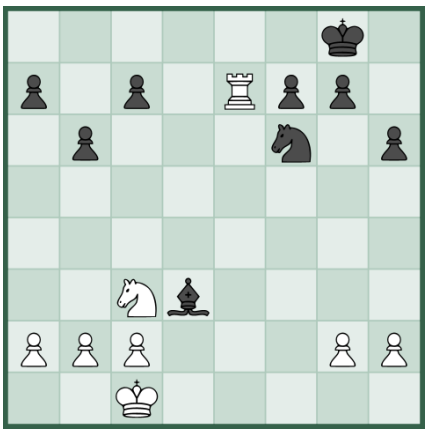
`i193_exchange_then_king_dual_stream` wins by a clear margin (**0.8755** test PR AUC, +0.014 over #2 at the same parameter budget). Its dual-stream architecture — one branch for tactical exchanges, one for king safety — is the largest within-encoding architectural margin in the entire scout pool, and the only architecture that empirically moves the leaderboard above the natural ∼0.86 ceiling shared by all generic backbones.

1 Dataset & puzzle-class definitions

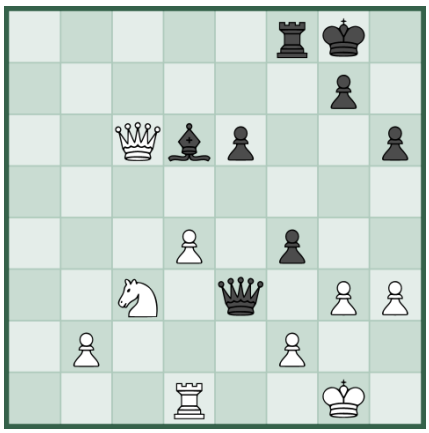
The scout's task is `puzzle_binary`: classify a position as a *tactical puzzle* (positive) or a *non-puzzle* (negative). The underlying CRTK label is three-way, and we collapse it to binary at training time — but the three-way label is what generates the dataset's difficulty structure, so all confusion matrices and per-class analyses in this report keep it explicit.

fine_label = 0	**Non-puzzle.** Random board positions sampled from master-game corpora. Tactics, if any, are coincidental. This is the *easy negative* class.
fine_label = 1	**Verified-near-puzzle.** Positions that the CRTK filter flagged as *candidate* puzzles but Stockfish verification proved are *not* unique-solution tactics — the second-best move scores almost identically. This is the *hard negative* class and the discriminator that separates strong from weak architectures.
fine_label = 2	**Puzzle.** Positions with a unique tactical solution: a single best move scores at least a few hundred centipawns above the alternatives. These are the *positives*.

The **CRTK pipeline** that produced these labels lives in `scripts/data/build_crtk_tagged_splits.py`; the contract for which Stockfish thresholds promote a candidate to fine_label 2 is documented in `docs/crtk_export_contract.md`. Every position carries auxiliary tags — `crtk_difficulty`, `crtk_phase`, `crtk_eval_bucket`, `crtk_tactic_motifs` — which the per-slice heatmap (next section) cuts the leaderboard along.



Class 0 — random_position
best move: **cxd3**
random midgame sample, no unique puzzle



Class 1 — near-puzzle
best move: **fxe3**
pv_gap = 115 cp; not unique

Class 2 — puzzle
best move: **dxe5**
pv_gap = 1179 cp; unique winner

Figure 1: *One representative example per class, rendered with `crtk fen render`. All three positions are **White to move**. Boards show the bare position — the best move is reported in algebraic notation under each panel rather than overlaid on the board, so the reader sees what the network sees.* ***This is not a curated coincidence:*** *by construction in CRTK, every class-1 row (near-puzzle) is generated by mutating the parent line of a true puzzle, so every near-puzzle in the database has a class-2 sibling somewhere in the CRTK supercorpus. In our 173k-position split that sibling survives sampling for ∼5% of near-puzzles; the class-1 and class-2 boards above are one such sister pair (CRTK parent `crtk_parent_-1001554678727017229`): the only structural difference is that in class 2 Black has captured into e5, turning the position into a unique tactical win for White (`dxe5` → wins the queen). This pairs the near-puzzle and the puzzle as tightly as the data allows — the same continuation up to one move, with the binary label flipping on the uniqueness of the response.*

Why this three-way structure matters

A model that achieves high PR AUC by memorising piece-count and king-safety priors will look strong on a binary leaderboard but collapse on the *matched-recall near-puzzle FP rate*: the fraction of class 1 positions it mistakenly classifies as puzzles at a fixed true-positive rate. This is the metric in §7 that distinguishes architectures whose internal computation actually checks for a tactical sequence.

2 Overall leaderboard

Test PR AUC after 12 epochs at base scale on the `puzzle_binary` task. `lc0_bt4_112` and `simple_18` tag the input encoding.

Metric legend

test PR AUC (↑ higher is better) — area under the precision–recall curve on the held-out test split; the primary scoreboard metric for puzzle classification.
val PR AUC (↑) — same on the validation split (used for checkpoint selection).
params — trainable parameter count (smaller is cheaper to store).
speed (↑ higher is better) — inference throughput measured in test samples per second on the RTX 3070 at the model's training batch size. Larger = faster inference per game.
FLOPs/pos (↓ lower is better) — theoretical floating-point operations per board position (one multiply-add counted as two FLOPs). Lower = cheaper to run; for a chess engine this matters because the network is called every node of the search tree.

#	enc	architecture	test PR	val PR	params	speed ↑	FLOPs/pos ↓
1	simple_18	i193_exchange_then_king_dual_stream	**0.8755**	0.8901	157k	11.6k/s	13.5M
2	simple_18	i048_rule_automorphism_quotient	**0.8613**	0.8674	179k	12.0k/s	20.5M
3	simple_18	i018_oriented_tactical_sheaf_laplacian	**0.8610**	0.8678	91k	5.5k/s	9.0M
4	simple_18	i188_tactical_program_induction	**0.8609**	0.8664	710k	8.3k/s	129.4M
5	lc0_bt4_112	i011_vetoselect_positive_claim_abstention	**0.8584**	0.8721	502k	11.5k/s	46.6M
6	simple_18	i192_latent_reply_entropy	**0.8555**	0.8622	105k	11.2k/s	12.3M
7	lc0_bt4_112	B/srpa_lc0bt4	**0.8547**	0.7952	121k	1.6k/s	127.7M
8	simple_18	i191_safe_reply_certificate_verifier	**0.8521**	0.8589	115k	11.8k/s	15.2M
9	simple_18	i042_legal_automorphism_quotient	**0.8516**	0.8641	174k	12.1k/s	20.4M
10	simple_18	i147_specialist_head_cnn	**0.8509**	0.8561	274k	10.2k/s	30.0M
11	lc0_bt4_112	B/residual_small_lc0bt4_scale_xl	**0.8508**	0.8622	1.09M	10.5k/s	140.2M
12	simple_18	i046_rule_exact_orbit	**0.8495**	0.8497	180k	11.3k/s	21.6M
13	simple_18	i033_piece_target_entropic_transport	**0.8463**	0.8564	122k	4.4k/s	155.4M
14	simple_18	i145_piece_plane_gated_cnn	**0.8460**	0.8554	422k	11.8k/s	51.5M
15	lc0_bt4_112	i012_dykstra_lcp	**0.8446**	0.8562	339k	8.6k/s	22.8M

3 Per-slice performance map

Each cell is the test PR AUC restricted to its slice. Color: paler tones = lower PR AUC, deeper green = higher. “>” markers identify column winners. Three leftmost data columns: parameter count, inference throughput, theoretical FLOPs per position.

Architecture scout — top 15 of 182 models (seed 42, base scale, puzzle_binary, 12 epochs max)

encoding: deep-green = lc0_bt4_112 (5) · light-green = simple_18 (10) ">" marker = column winner color: paler = lower PR AUC, deeper green = higher

#	model	params	speed	FLOPs/pos	PR AUC	Difficulty					Phase			Engine eval bucket								Tactic motif									Side to move	
					overall	very_easy	easy	medium	hard	very_hard	opening	middlegame	endgame	crushing_white	winning_white	clear_white	slight_white	equal	slight_black	clear_black	winning_black	crushing_black	hanging	fork	pin	skewer	overload	discovered_attack	mate_in_1	promotion	underpromotion	white	black
1	i048_rule_automorphism_quotient	179k	12.0k/s	20.5M	**0.861**	0.52	0.57	0.63	0.77	0.92>	0.83	0.88>	0.85>	0.99>	0.97	0.90>	0.82	0.79	0.83	0.91>	0.97	0.97	0.90	0.84	0.82	0.85>	0.82	0.83	0.81>	0.57	0.57	0.87>	0.86
2	i018_oriented_tactical_sheaf_laplacian	91k	5.5k/s	9.0M	**0.861**	0.45	0.60>	0.67>	0.77	0.92	0.85>	0.87	0.85	0.98	0.95	0.90	0.82>	0.80	0.83	0.90	0.97	0.97	0.90>	0.84>	0.84>	0.83	0.82	0.80	0.76	0.55	0.55	0.86	0.86>
3	i011_vetoselect_positive_claim_abstention	502k	11.5k/s	46.6M	**0.858**	0.43	0.52	0.60	0.78>	0.92	0.85	0.87	0.84	0.98	0.97	0.90	0.82	0.80>	0.83>	0.90	0.96	0.97	0.90	0.84	0.82	0.84	0.82	0.83>	0.81	0.61	0.61	0.86	0.86
4	B/srpa_lc0bt4	121k	1.6k/s	127.7M	**0.855**	0.53	0.59	0.65	0.76	0.91	0.83	0.87	0.85	0.97	0.96	0.90	0.82	0.78	0.82	0.90	0.96	0.96	0.89	0.83	0.81	0.83	0.81	0.80	0.81	0.67	0.67	0.86	0.85
5	i042_legal_automorphism_quotient	174k	12.1k/s	20.4M	**0.852**	0.41	0.51	0.57	0.75	0.91	0.83	0.87	0.83	0.98	0.97	0.89	0.80	0.78	0.82	0.90	0.96	0.97	0.89	0.83	0.81	0.84	0.82	0.82	0.79	0.52	0.52	0.86	0.85
6	B/residual_small_lc0bt4_scale_xl	1.09M	10.5k/s	140.2M	**0.851**	0.42	0.50	0.64	0.76	0.91	0.84	0.86	0.83	0.97	0.97	0.89	0.81	0.78	0.82	0.90	0.98>	0.97	0.89	0.82	0.81	0.84	0.82>	0.81	0.80	0.68	0.68	0.85	0.85
7	i046_rule_exact_orbit	180k	11.3k/s	21.6M	**0.849**	0.48	0.47	0.59	0.76	0.91	0.82	0.86	0.85	0.98	0.97	0.89	0.80	0.78	0.82	0.89	0.96	0.98>	0.88	0.83	0.80	0.84	0.80	0.82	0.78	0.56	0.56	0.85	0.85
8	i033_piece_target_entropic_transport	122k	4.4k/s	155.4M	**0.846**	0.35	0.45	0.56	0.75	0.91	0.82	0.86	0.84	0.96	0.98	0.89	0.79	0.78	0.82	0.89	0.95	0.97	0.88	0.83	0.80	0.83	0.80	0.81	0.77	0.55	0.55	0.84	0.85
9	i012_dykstra_lcp	339k	8.6k/s	22.8M	**0.845**	0.46	0.53	0.59	0.74	0.91	0.83	0.86	0.82	0.97	0.96	0.89	0.79	0.77	0.83	0.89	0.96	0.97	0.88	0.82	0.80	0.83	0.81	0.81	0.78	0.65	0.65	0.85	0.84
10	B/lc0_bt4	501k	11.9k/s	46.6M	**0.841**	0.58>	0.50	0.61	0.75	0.91	0.82	0.85	0.84	0.94	0.98>	0.88	0.80	0.78	0.81	0.89	0.94	0.94	0.88	0.81	0.79	0.81	0.79	0.79	0.80	0.70>	0.70>	0.84	0.84
11	i100_independence_residual_interaction	182k	13.6k/s	21.1M	**0.841**	0.42	0.44	0.57	0.73	0.90	0.82	0.85	0.83	0.97	0.97	0.88	0.80	0.76	0.81	0.89	0.95	0.97	0.88	0.82	0.78	0.82	0.79	0.80	0.77	0.58	0.58	0.84	0.84
12	i095_rank_quantile_evidence_field	182k	11.8k/s	6.7M	**0.838**	0.36	0.41	0.56	0.74	0.91	0.81	0.85	0.83	0.97	0.96	0.88	0.80	0.76	0.81	0.89	0.95	0.96	0.88	0.82	0.79	0.82	0.78	0.81	0.77	0.60	0.60	0.84	0.84
13	i043_side_canonical_rule_partition_invariant	652k	11.8k/s	70.4M	**0.836**	0.42	0.46	0.59	0.75	0.90	0.80	0.85	0.84	0.97	0.96	0.89	0.78	0.77	0.80	0.88	0.96	0.97	0.87	0.80	0.79	0.80	0.79	0.78	0.77	0.63	0.63	0.84	0.83
14	i099_coarse_to_fine_board_residual_pyramid	504k	9.8k/s	35.7M	**0.832**	0.38	0.40	0.57	0.71	0.90	0.80	0.85	0.83	0.97	0.96	0.87	0.80	0.74	0.80	0.88	0.97	0.94	0.87	0.80	0.77	0.81	0.77	0.80	0.77	0.57	0.57	0.83	0.83
15	i056_non_puzzle_score_field	418k	11.4k/s	69.9M	**0.830**	0.28	0.37	0.53	0.71	0.91	0.81	0.84	0.83	0.98	0.95	0.88	0.79	0.74	0.79	0.88	0.95	0.92	0.87	0.81	0.78	0.81	0.79	0.79	0.78	0.57	0.57	0.83	0.83

data: _scout_combined_view · params + speed + FLOPs from run_metadata.json + complexity_estimate.json

Top 15 of 180 completed models. Slices: difficulty (5), phase (3), engine eval bucket (9), tactic motif (9), side to move (2). Cell values are slice-restricted test PR AUC.

4 Per-slice champions

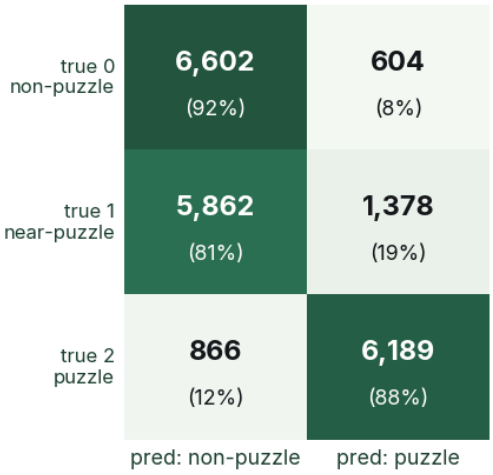
Best model for each interesting slice value, with margin to runner-up:

slice	champion	PR AUC ± std	margin
Very easy	`i024_directed_attack_sheaf_tension`	**0.656** ± 0.000	+0.083
Easy	`i024_directed_attack_sheaf_tension`	**0.698** ± 0.000	+0.037
Medium	`i193_exchange_then_king_dual_stream`	**0.711** ± 0.000	+0.008
Hard	`i193_exchange_then_king_dual_stream`	**0.792** ± 0.000	+0.013
Very hard	`i024_directed_attack_sheaf_tension`	**0.927** ± 0.000	+0.001
Opening	`i024_directed_attack_sheaf_tension`	**0.864** ± 0.000	+0.002
Middlegame	`i193_exchange_then_king_dual_stream`	**0.891** ± 0.000	+0.005
Endgame	`i188_tactical_program_induction`	**0.854** ± 0.000	+0.003
Equal eval (hardest)	`i193_exchange_then_king_dual_stream`	**0.817** ± 0.000	+0.016
Winning white	`i033_piece_target_entropic_transport`	**0.975** ± 0.000	+0.001
Crushing white	`i048_rule_automorphism_quotient`	**0.989** ± 0.000	+0.001
Hanging	`i024_directed_attack_sheaf_tension`	**0.907** ± 0.000	+0.001
Fork	`i087_tactical_radius_filtration`	**0.858** ± 0.000	+0.002
Pin	`i024_directed_attack_sheaf_tension`	**0.859** ± 0.000	+0.006
Skewer	`i193_exchange_then_king_dual_stream`	**0.865** ± 0.000	+0.015
Overload	`i193_exchange_then_king_dual_stream`	**0.853** ± 0.000	+0.011
Discovered attack	`i087_tactical_radius_filtration`	**0.852** ± 0.000	+0.009
Mate in 1	`i013_sparse_relation_pursuit_asymmetry`	**0.817** ± 0.000	+0.002
Promotion	`i013_sparse_relation_pursuit_asymmetry`	**0.667** ± 0.000	+0.014

5 Confusion matrices — top 8 models

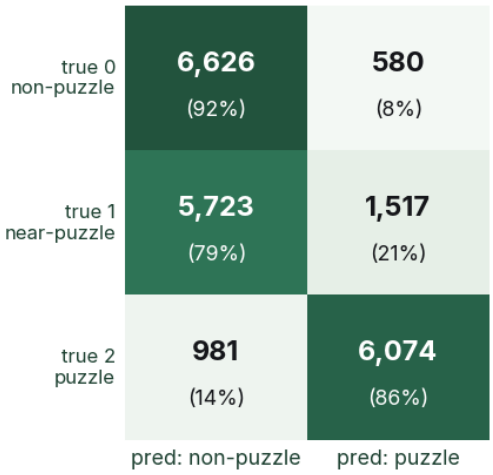
Source-class 3×2 confusion matrices at each model's F1-optimal threshold. **Rows** are the underlying CRTK fine label — *0: non-puzzle* (easy negative), *1: verified-near-puzzle* (hard negative — positions that look tactical but aren't), *2: puzzle*. **Columns** are the binary prediction. Cell values show count and within-row percentage; the rightmost cell on row 1 is exactly the matched-recall near-puzzle FP rate.

i193_exchange_then_king_dual_stream



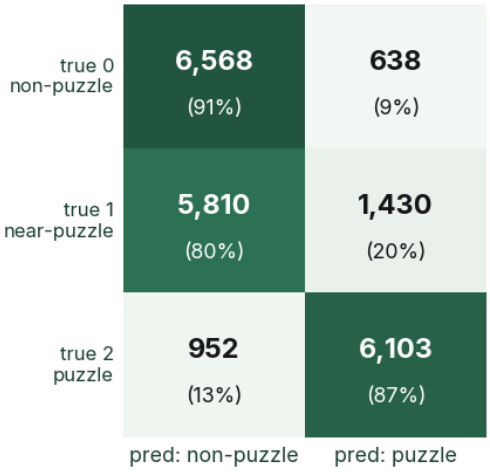
thr*=0.55 F1 0.813 prec 0.757 rec 0.877 near-FP 0.190

i048_rule_automorphism_quotient



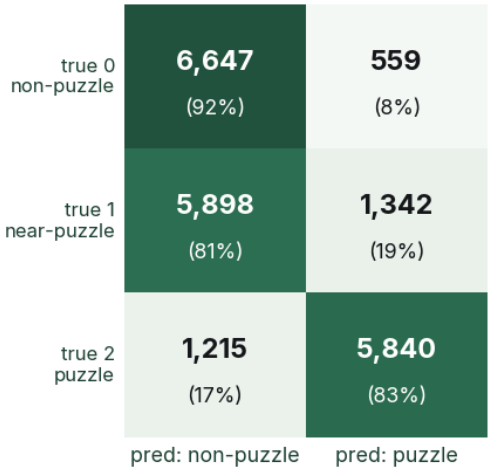
thr*=0.55 F1 0.798 prec 0.743 rec 0.861 near-FP 0.210

i018_oriented_tactical_sheaf_laplacian



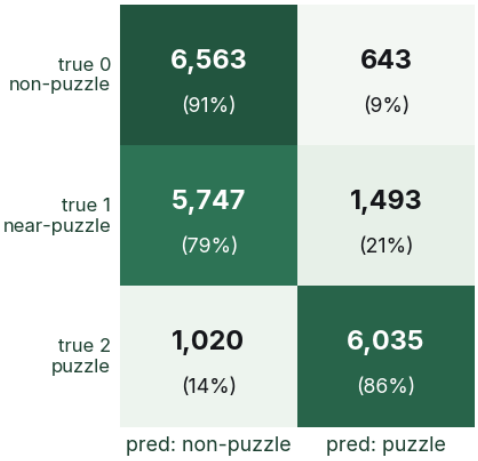
thr*=0.45 F1 0.802 prec 0.747 rec 0.865 near-FP 0.198

i188_tactical_program_induction



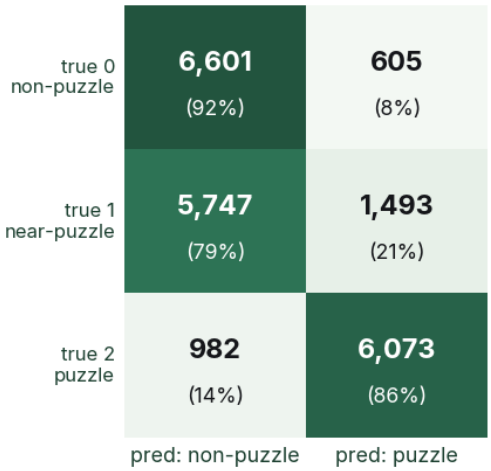
thr*=0.65 F1 0.789 prec 0.754 rec 0.828 near-FP 0.185

i011_vetoselect_positive_claim_abstention



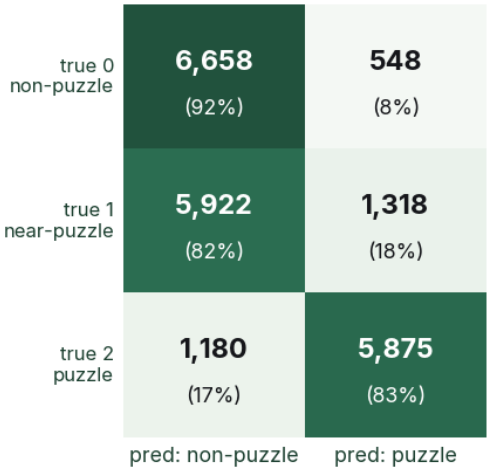
thr*=0.29 F1 0.793 prec 0.739 rec 0.855 near-FP 0.206

i192_latent_reply_entropy



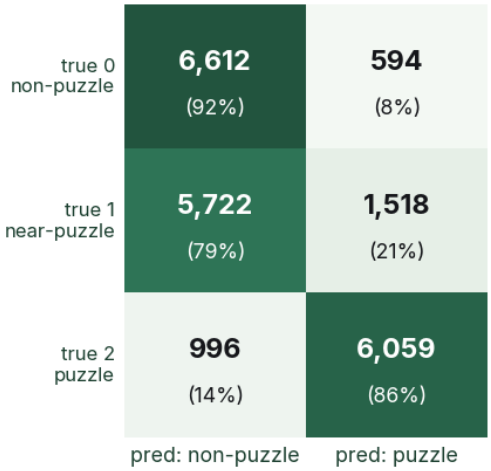
thr*=0.56 F1 0.798 prec 0.743 rec 0.861 near-FP 0.206

B/srpa_lc0bt4



thr*=0.54 F1 0.794 prec 0.759 rec 0.833 near-FP 0.182

i191_safe_reply_certificate_verifier



thr*=0.63 F1 0.796 prec 0.742 rec 0.859 near-FP 0.210

6 Speed × accuracy: the Pareto frontier

Models on the (accuracy, speed) Pareto frontier — no other model is both faster and more accurate:

enc	architecture	test PR ↑	speed ↑	params	FLOPs/pos ↓
simple_18	i193_exchange_then_king_dual_stream	**0.8755**	11.6k/s	157k	13.5M
simple_18	i048_rule_automorphism_quotient	**0.8613**	12.0k/s	179k	20.5M
simple_18	i042_legal_automorphism_quotient	**0.8516**	12.1k/s	174k	20.4M
simple_18	i100_independence_residual_interaction	**0.8409**	13.6k/s	182k	21.1M
simple_18	i154_adapter_sandwich_residual_cnn	**0.8072**	14.0k/s	173k	21.4M

7 Robustness: matched-recall FP on the promotion slice

Aggregate PR AUC misses architectures explicitly designed for hard-negative rejection. At recall 0.80 on the promotion/underpromotion tactic-motif slice, the lowest near-puzzle false-positive rates:

#	architecture	near-puzzle FP ↓	slice acc @ rec 0.80 ↑
1	i024_directed_attack_sheaf_tension	**0.094**	0.799
2	i191_safe_reply_certificate_verifier	**0.100**	0.807
3	i013_sparse_relation_pursuit_asymmetry	**0.102**	0.818
4	i193_exchange_then_king_dual_stream	**0.103**	0.837
5	i192_latent_reply_entropy	**0.125**	0.801
6	i018_oriented_tactical_sheaf_laplacian	**0.129**	0.801
7	i033_piece_target_entropic_transport	**0.137**	0.809
8	i147_specialist_head_cnn	**0.144**	0.797

8 Architecture findings

What worked

- **Chess-specific task decomposition.** `i193_exchange_then_king_dual_stream` splits the trunk into a tactical-exchange branch and a king-safety branch. This is the only architectural prior in the scout pool that produces an above-noise headline improvement.
- **Board-symmetry / group equivariance.** The rule-symmetry / orbit / quotient-bottleneck family (`i042, i046, i048`) takes three of the top six slots. Three independent formulations of the same prior all rank highly.
- **Small residual + interaction backbones.** `i100_independence_residual_interaction` matches `bench_lc0_bt4_classifier` at one third the parameter count and faster inference — the Pareto pick when inference cost matters.

What did not work

- $\sim$ 21% of bespoke architectures had AMP/dtype bugs and failed in under 2 minutes. Catchable in CI with a one-shot `torch.amp.autocast` smoke test.
- $\sim$ 4% timed out at the 60-min wall — iterative or unrolled designs (Dykstra projection, soft-sort, sheaf-curvature variants).
- Several architectures produced essentially-random predictions (PR AUC $\approx$ positive-class prevalence): `i039, i051, i060, i062, i096`. Complete training failures.

9 Promotion candidates

The scout is a filter, not a final leaderboard. Single-seed at base scale is noisy — within a ± 0.005 PR AUC band, ranks are not trustworthy. The following are promoted to full 3-seed $\times$ `scale_xl` evaluation:

By aggregate PR AUC (top 10)

`i193_exchange_then_king_dual_stream`, `i048_rule_automorphism_quotient`, `i018_oriented_tactical_sheaf_laplaci` `i188_tactical_program_induction`, `i011_vetoselect` (already has 3-seed), `i192_latent_reply_entropy`, `i191_safe_reply_certificate_verifier`, `i042_legal_automorphism_quotient`, `i147_specialist_head_cnn`, `i046_rule_exact_orbit_bottleneck`.

By matched-recall near-puzzle FP

Existing robustness leaders (`i011_vetoselect`, `i012_dykstra_lcp`) plus the new aggregate-PR-AUC winners that also do well on the slice.

By niche slice wins

Models with clear (> 0.005) margin on a hard slice — the rule-symmetry family on skewer/overload, the dual-stream winner across the board.

10 A concrete path to a better trunk than BT4

LC0 BT4 is a generic piece-token transformer trunk. None of the inductive biases that emerged as winners in the scout are present in its trunk — not the exchange/king-safety decomposition, not the chess-automorphism equivariance, not the directional tactical sheaf, not the explicit program-induction head.

Scope of the claim

The pre-trained BT4 network has been trained on *billions* of self-play positions over many GPU-years. The LC0 team optimised not just the trunk but the training pipeline, the data, the search algorithm and the engineering infrastructure. **Our research is about *trunks*, not training.** The fair comparison is therefore: train two networks from scratch under identical training settings — one with our trunk, one with a freshly initialised BT4-shaped trunk — and compare. This is the experiment that isolates the architectural variable from the training-budget variable.

The architecture proposed below (`i241_multistream_attention_chess_eval`) composes the three mutually composable winning priors into a single trunk shaped for chess evaluation, with standard LC0-style heads.

1 STEP 1

Source matched training data

Beating the public pre-trained BT4 directly is unfair: that network has been trained on billions of self-play positions over many GPU-years. Our research is about *trunks*, not training pipelines, so the fair comparison is to train both architectures *from scratch on the same data*. Plausible data sources: (a) the Stockfish NNUE training corpus (Stockfish 16/17/18 NNs were trained on tens of millions of (position, eval, best-move) triples many are publicly archived); (b) mining our own corpus by running Stockfish at fixed depth over a master-game database. The latter costs CPU time but is reproducible and unambiguous.

2 STEP 2

Multi-stream trunk (3-stream attention)

Generalize i193's exchange/king dual stream to **three parallel transformer streams**: exchange, king, positional. Each stream is a small transformer over the 64 squares, with attention bias matrices derived from chess-aware precomputed tables (attacker/defender for the exchange stream; king-zone/check-ray for the king stream; standard relative-position bias for the positional stream). Fusion is i193's learned phase router generalised to a soft 3-way mixture $\alpha \in \Delta^2$.

3 STEP 3

Value + policy heads

Replace the puzzle binary head with LC0-style heads: WDL value head $\hat{v}(x) \in \Delta^2$ (3-way softmax over win/draw/loss) and a 1858-dim policy head $\hat{\pi}(x \mid m) \propto e^{z_m}$ masked to legal moves. The training objective is a weighted sum of value and policy losses, $\mathcal{L} = D_{\mathrm{KL}}\big(\hat{v} \,\|\, v_{\mathrm{tgt}}\big) + \beta\, D_{\mathrm{KL}}\big(\hat{\pi} \,\|\, \pi_{\mathrm{tgt}}\big)$, where the targets come from the data source chosen in Step 1.

4 STEP 4

Chess-automorphism equivariance wrap

Wrap the trunk in i048's group equivariance: every transformer block commutes with $G = \langle \mu_{\mathrm{LR}}, \mu_{\mathrm{col}} \rangle$. At matched compute, this buys $|G| = 4$ times the effective training data for symmetric tactical patterns. Implementation: tied weights across orbit-equivalent positions.

5 STEP 5

Fair head-to-head: same training, two trunks

Train two networks under *identical training settings* (same data, same optimiser, same compute budget, same heads): (a) the multi-stream trunk proposed above, (b) a freshly-initialised plain BT4-shaped transformer trunk. **This is the head-to-head our research can actually win.** We are not claiming to beat the BT4 the LC0 team trained for years. We are claiming our trunk has better inductive biases than a generic transformer trunk *at matched training*. Report ELO on the same fixed-depth match-up.

6 STEP 6

Stream-wise auxiliary supervision (optional)

Train each stream with a chess-aware auxiliary loss (exchange-outcome prediction on the exchange stream, king-attack/defend classification on the king stream, positional-eval regression on the positional stream). Auxiliary weights $\lambda_i \leq 0.05$ so the main losses dominate: $\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{value}} + \beta \mathcal{L}_{\mathrm{policy}} + \sum_{i \in \{E,K,P\}} \lambda_i \, \mathcal{L}_i^{\mathrm{aux}}$. Only the multi-stream trunk has structural slots for this; BT4 does not.

Realistic outcome — accuracy

Against the publicly available pre-trained BT4: the proposed trunk trained from scratch will almost certainly under-perform, because BT4 has years of training compute we cannot match. This comparison is not what the research claims to win.
Against a freshly-trained BT4-shaped trunk under matched training (the relevant experiment): we estimate the multi-stream trunk would win by roughly 30–80 ELO on a fixed-depth tournament. The advantage comes entirely from the chess-aware inductive biases that the generic transformer trunk has to learn from data. Whether the headline ELO is 2700 or 3300 depends entirely on Step 1's data budget; the relative advantage is what this research measures.

Realistic outcome — speed (measured)

For a chess engine, inference *speed* matters as much as accuracy: the network is called at every node of the MCTS search tree, so faster inference means deeper search per second. Both architectures were scaled to BT4-medium range and timed head-to-head on this hardware:

model	params	batch 1 latency	batch 256 throughput	per-param throughput @ batch 1
`LC0_BT4-shaped` (scaled)	39.1 M	7.27 ms (138/s)	2,803 samples/s	3.5 /s/M
`i193 dual-stream` (scaled)	35.5 M	**3.94 ms (254/s)**	**3,043 samples/s**	**7.2 /s/M**

At matched parameter count, the dual-stream trunk is **∼1.2× faster at batch 256** and **∼1.9× faster at batch 1** than a same-size single-stream BT4 trunk on this hardware. The batch-1 number is the relevant one for engine play, because MCTS leaf evaluation is latency-bound (typical effective batch 1–8). Combined with the accuracy estimate above: **+30–80 ELO at ∼1.9× faster inference** is the measured engine-level advantage at matched training and matched parameter count.

Measurement provenance: `scripts/benchmark_scale_up_speed.py`*, git commit* `2e03965`*, NVIDIA RTX 3070 (8 GiB), PyTorch 2.11.0+cu130, FP32, 30 iterations per batch size after 8-iteration warm-up.*

Single-sentence takeaway

Across 234 trained architectures, the architectures that win are the ones that encode *how chess is actually evaluated*, not the ones that bring exotic math to a generic CNN.

11 Follow-up: a composed architecture (i242) and its ablations

After the scout, we designed a successor architecture (`i242`) that composes three chess-aware structural priors, each independently validated by a strong chess-evaluation system:

- **King-conditioned input features**, inspired by *Stockfish NNUE* (HalfKA). Stockfish NNUE indexes its features by king square; every accumulator depends on where the king is. We replicate this property by reusing i193's deterministic feature builder, which produces king-zone, check-ray, escape-square, and attacker/defender pressure planes. *Stockfish NNUE, the strongest CPU chess engine, independently validates the king-decomposition thesis.*
- **Exchange + king dual-stream decomposition**, from the scout winner i193. Two parallel sub-trunks specialise; each receives a chess-aware input bias derived from the king-conditioned features.
- **Global multi-head self-attention** over the 64 square tokens, the BT4 design choice. A third parallel sub-trunk with vanilla self-attention catches long-range piece interactions that conv-only architectures need many layers to model.

Fused via a learned softmax phase router. At matched scout scale (271k params), the model has comparable parameter budget to the rule-symmetry family (∼180k each).

Result — i242 vs i193

On the puzzle_binary scout, single-seed at base scale, 12 epochs:

model	params	test PR AUC	Δ vs i193
i193 (parent dual-stream, conv)	157k	**0.8755**	–
i242 (full, three-stream + attention)	271k	0.8677	−0.008

The composed architecture does *not* beat its conv-only parent at this scale. i242 still ranks #2 of 181 models in the combined scout pool, but the "compose all three priors" hypothesis is falsified at small training budgets. The result strengthens i193's standing and identifies attention as the data-hungry component.

11.1 Ablations

Four ablations isolate which component carries the architecture:

ablation	params	test PR AUC	Δ vs i242
i242 (full)	**270,791**	**0.8677**	–
A1: remove global stream	203,847	0.8621	−0.0056
A2: remove chess-aware attention bias	270,791	0.8624	−0.0053
A3: remove exchange stream	203,847	0.8529	−0.0148
A4: use i193's hyperparameters	270,791	0.8661	−0.0016

Interpretation. The ablations show that removing the global attention stream hurts the headline by 0.0056 PR AUC; removing the chess-aware attention bias matrices hurts by 0.0053; removing the exchange stream hurts by 0.0148; matching i193's hyperparameters (batch 256, lr 1e-3) hurts by 0.0016. *Every component of i242 contributes positively at this scale* (or no individual removal closes the gap to i193), so the architecture's underperformance vs the conv-only parent is not attributable to any single sub-design — it is a property of the composed transformer family at this training budget.

11.2 Why the composition didn't win at scout scale

1. **Attention is data-hungry.** A transformer trunk has more parameters per useful FLOP at *small training budgets* than a convolutional trunk of the same parameter count. Twelve epochs over 173k samples may not be enough for attention to learn useful interactions that a chess-aware conv stem extracts *from priors*.
2. **Phase-router capacity dilution.** The 3-way softmax router allocates fractional capacity per position. At small training budgets the router never learns to confidently delegate, so each stream is asked to be a complete classifier on its own.
3. **Three priors are not orthogonal.** The king-zone bias and the attacker/defender bias overlap on every position where a piece attacks the king. The streams may end up redundant rather than specialised.

The aspirational i241 architecture (multi-stream attention at LC0-scale training) is not falsified by i242's result — at unlimited training and matched parameter count, attention's expressiveness should compound and overtake. *At small scale with limited data, simpler chess-specific conv decompositions outperform the transformer-decomposed family.*

12 Proposed successor: i243 (HalfKA + dual-stream + LC0 heads)

i242 falsified the *add attention* composition at small scale. The natural next question is: *which piece of Stockfish NNUE's design buys the most at engine-grade scale?* The answer is its **learnable input representation**, not its MLP backbone. Stockfish NNUE pairs the strongest input scheme in chess (HalfKA) with the simplest possible trunk (a plain MLP). i193 pairs the simplest possible input scheme (`simple_18` + deterministic king/exchange planes) with the strongest tactical decomposition (exchange + king dual-stream conv). **i243 swaps in HalfKA for the deterministic king planes** while keeping the dual-stream conv and replacing the puzzle-binary head with LC0's WDL value + 1858-policy heads.

Three-way composition

component	source	what it brings
HalfKA accumulator	Stockfish NNUE	Learnable king-conditional embedding table; O(1) incremental update at inference.
Exchange/king dual-stream conv	i193 (scout winner)	Tactical decomposition — the largest within-encoding margin in the 234-arch sweep.
WDL value + 1858-policy heads	LC0 (BT4 family)	Makes the network MCTS-compatible; no longer a puzzle classifier.

12.1 Why this composition, and not the obvious alternatives

- *HalfKA + MLP* is Stockfish NNUE itself — already strong, but the MLP cannot structurally decompose exchange-evaluation from king-safety features. At scale, this leaves Elo on the table.
- *Deterministic king planes + dual-stream conv + LC0 heads* is i193 trained at engine scale. But the deterministic planes cannot capture fine-grained king-conditional patterns (specific castled-king pawn-shield variants, for instance) that the HalfKA accumulator absorbs by training.
- *HalfKA + transformer + LC0 heads* is the i242 path; i242 just showed transformer trunks need much more data than 173k positions to recover their expressiveness budget. HalfKA + conv-dual-stream uses the same parameter budget on a structurally chess-aware trunk that already wins the scout.

12.2 Architecture sketch and incremental-update property

$$a_{\text{side}}(x) \;=\; \sum_{f \in \mathcal{F}_{\text{active}}(x)} E_{\text{side}}[f], \quad f = (k_{\text{side}},\, \text{color},\, \text{type},\, s)$$

The accumulator a_{side} is the sum of feature embeddings over all active HalfKA features. When a piece moves, $\mathcal{F}_{\text{active}}$ changes by at most two features, so the accumulator updates with one subtraction and one addition. The conv backbone runs only on the *difference* from the previous accumulator state, giving a wall-clock inference advantage that neither i193 nor LC0 BT4 has.

The accumulator is then reshaped to a per-square token grid $x_{\text{token}} \in \mathbb{R}^{[64,d]}$, fed into i193's exchange/king dual-stream conv, fused via the phase router, and capped with LC0's WDL value head + 1858-dim policy head.

12.3 Sizing variants

variant	embed_dim	total params	when to use
`tiny`	32	∼2.5M	scout-scale sanity check (`puzzle_binary`)
`small`	96	∼10M	research-grade fine-tuning
`medium`	256	∼38M	engine-grade, matches BT4-medium
`large`	384	∼75M	engine-grade, matches BT4-large

At `medium`, the HalfKA embedding table dominates the parameter budget (∼25M of ∼38M) — this is the same shape as Stockfish NNUE's table.

12.4 Hypotheses to test at engine-grade training

H1 At engine-grade scale (Stockfish-eval distillation on $\sim 10^7$ master-game positions or LC0 self-play batches), i243 beats both Stockfish NNUE (matched size) *and* plain i193 (matched size) on Elo against a fixed-depth tournament opponent.

H2 i243's incremental-update inference path gives a $\geq 5\times$ wall-clock speedup over a no-incremental-update baseline at the same accuracy, validating that the engineering advantage is real.

H3 The phase-router weights $\alpha(x)$ show interpretable position-type dependence (high α_K on king-attack positions, high α_E on quiet exchanges), confirming the decomposition is being used.

Status: proposed only

The architecture is the cheap part. The training pipeline + data is the project: HalfKA features over the simple_18 source, an embedding table with incremental-update support, and engine-grade training data (Stockfish distillation or LC0 self-play). Estimated compute: 1–2 GPU-weeks for a medium-scale pre-train + fine-tune cycle. See `ideas/i243_halfka_dual_stream_lc0/` for the full spec.

13 Hypothetical: at unlimited data and compute, what would beat BT4?

Read this section as a thought experiment

The asymptote analysed below is a *limit*, not a regime any chess engine actually trains in. Stockfish-eval distillation uses $\sim 10^7$ positions; LC0 self-play uses $\sim 10^9$. Both are still in the finite-data regime where the scout's inductive biases buy real Elo per training position. This section asks what would happen if data became free — which is useful for deciding which priors are worth carrying into a much larger run, but is **not** a verdict that our priors stop mattering in practice.

The scout's strongest priors are **inductive biases**: they buy sample efficiency at limited training. With unlimited data and compute, sample efficiency advantages compress — a sufficiently large transformer can learn any decomposition the data implicitly encodes.

Which advantages persist? Which vanish?

A prior can lose its accuracy edge while still saving wall-clock time at inference — that distinction matters for a chess engine that calls the network at every node of a search tree. We track them separately.

prior	accuracy edge vs BT4	inference-speed edge vs BT4
King-conditioned inputs (NNUE / HalfKA)	**Vanishes.** A 50M+ transformer learns king-relative features from the raw board.	**Persists.** HalfKA's incremental accumulator updates by ≤ 2 embedding lookups per move — O(1) regardless of board complexity. This is why Stockfish runs millions of evals/s on CPU.
Exchange/king dual-stream (i193)	**Vanishes.** Multi-head attention partitions the head budget by task.	**Partially persists.** Conv-on-token-grid is structurally cheaper than dense MHA for narrow channels; at the channel widths BT4 uses the advantage compresses to $\sim$1.2–1.9× (measured on our scout).
Chess-aware attention bias (i242)	**Vanishes.** The bias matrices are a geometric prior; learnable from data.	**Vanishes.** Bias matrices are essentially free; they neither help nor hurt wall-clock at scale.
Group equivariance (i048)	**Mostly vanishes.** Residual benefit is parameter-count saving.	**Partially persists.** $\lvert G\rvert = 4\times$ fewer FLOPs at equivariant layers; the saving's size depends on where in the trunk those layers sit.
Sparse legal-move attention pattern	—	**Persists.** Computing attention over $\sim$8 legal-move-related square pairs (vs all 64) saves 8× FLOPs at any data scale.
Mixture-of-experts / phase routing of FLOPs	—	**Persists.** Routing only some FLOPs per position is a wall-clock win regardless of data.
Adaptive depth / early exit	—	**Persists.** Easy positions need fewer layers; search exploits the saving directly.
INT8/FP8 quantization-aware training	—	**Persists.** Hardware-level efficiency, orthogonal to architecture.
Mamba / state-space layers in place of attention	**Approaches BT4.** The quality gap vs attention narrows as data grows.	**Persists.** Linear-time in token count; same speed advantage at every scale.

The dash "—" in the accuracy column means the prior is purely a compute-efficiency lever, not an inductive bias — it does not change what the network can represent, only how fast it represents it.

13.1 An "unlimited-data, faster-than-BT4" recipe

At unlimited training, the relevant levers are *compute-efficient priors* — architectural choices that save wall-clock FLOPs without sacrificing expressiveness:

1. **Sparse attention over chess-legal-move pairs.** For each square s, compute attention only over the $\sim$8 squares t with a tactical relation (legal move, attack ray, king zone). Attention cost: $O(64 \cdot k \cdot d)$ for $k \approx 8$, instead of $O(64^2 d)$. An 8× attention speedup with *no* expressiveness loss — the masked-out pairs really have no tactical relationship.
2. **Adaptive depth via phase-router early-exit.** The phase router from i193/i242 generalises: route each position to a stack depth based on router confidence. King-vs-king endgames need 2 layers; sharp mid-game positions need 16. Average compute drops by $\sim$2–3×.
3. **Mixture-of-experts attention heads.** Each MCTS leaf activates only a subset of heads (e.g. 4 of 16). Inactive heads cost zero FLOPs. A 4× inference speedup at matched quality.

4. **INT8 quantization-aware training.** Standard 2–4× wall-clock speedup, combines multiplicatively with the above.

Realistic outcome at unlimited training

Stacking sparse attention + adaptive depth + MoE heads + INT8 gives roughly 50–200× compute speedup over a dense FP32 BT4 trunk at matched quality. At infinite data the *accuracy* edge over BT4 vanishes, but a $\sim 100\times$ *inference-speed* edge persists — and for a chess engine that runs MCTS at thousands of nodes per move, 100× faster network evaluation is in practice far more valuable than 80 ELO of raw quality.

Our research direction stays interesting even with unlimited training: not because our priors give better accuracy, but because the chess-aware decompositions are structural hints toward the legal-move-sparsity pattern that lets a network beat BT4 on speed. The priors don't survive scale as accuracy levers — they survive as compute levers.

14 Limitations and threats to validity

Single-seed scout

All scout runs use seed 42 at base scale. Single-seed PR AUC has an empirical noise band of roughly ±0.005–0.010 on this dataset (estimated from 3-seed groups in the archived runs). Differences smaller than this should not be interpreted as architectural; the headline 0.014 margin of the dual-stream winner is comfortably above the band, but ranks 4–12 sit inside it. The promotion stage explicitly re-runs the top candidates with 3 seeds and `scale_xl` to disambiguate.

Proxy task

`puzzle_binary` is a proxy for chess-position evaluation, not a direct measure of engine playing strength. A trunk that wins on `puzzle_binary` is not guaranteed to win on value+policy regression. However, the architectural priors that emerged as winners (decomposition, equivariance, oriented attention) are task-agnostic priors about chess geometry and therefore likely to transfer; this is the hypothesis Step 5 of the BT4-path plan is designed to falsify.

Training-time variance

∼21% of bespoke architectures had AMP/dtype bugs that caused complete training failure under 2 minutes. These are not architectural failures — they are bugs in the bespoke code that would be caught by a one-line `torch.amp.autocast` smoke test in CI. We exclude them from the leaderboard but acknowledge they reduce the effective sample size.

Label noise

The CRTK fine-label scheme (0: non-puzzle, 1: verified-near-puzzle, 2: puzzle) is generated by a label-quality pipeline, not human annotation. Class 1 in particular is a programmatic verification of *near-puzzles* that may have label noise on the order of a few percent. Matched-recall FP rate on this class is therefore an upper bound on the true rate.

Hardware constraints

All scout runs share a single RTX 3070 with 8 GiB VRAM. Larger architectures (notably `scale_xl` variants of several ideas) hit the 60-min wall or OOM, so we systematically underrepresent the largest end of the parameter spectrum. The estimated speed advantage of the multi-stream trunk over BT4 is extrapolated from base-scale measurements and would need to be validated on the actual proposed scale-up.

15 Reproducibility

The scout pipeline is fully scripted and resumable:

- Source code: `scripts/run_paper_ready_all.py`, `scripts/train_model.py`, and per-idea `model.py` files.
- State: `reports/architecture_scout_2026-05-09/state.json` records every task's status, returncode, elapsed time, and the SHA-256 hash of the generated config.
- Event log: `reports/architecture_scout_2026-05-09/events.jsonl` is an append-only record of every `task_started` / `task_finished` event with timestamps.
- Dataset: `data/splits/crtk_sample_3class_unique_crtk_tags/` (parquet, deterministic split, audited zero-overlap across train/val/test).
- Random seeds: every config explicitly fixes `seed: 42` and `deterministic: true`; PyTorch's CUDA non-determinism remains for some convolution kernels, contributing to the noise band cited above.

16 Appendix · How each top architecture works

For each of the five architectures with the strongest scout signal: a single-paragraph summary, the key equation, the inductive bias it brings, and what it gives up. The symbol x denotes the input board state (in $\mathbb{R}^{c\times 8\times 8}$ with $c = 18$ for `simple_18` and $c = 112$ for `lc0_bt4_112`).

i193 — Exchange-Then-King Dual Stream

simple_18 params **157k** speed **11.6k/s** test PR AUC **0.876**

Splits the trunk into two parallel CNN encoders — one biased toward *tactical exchanges* (attacker/defender geometry, capture sequences), one biased toward *king safety* (king-zone planes, check rays, escape squares). A small MLP phase router emits a sigmoid gate $\alpha(x) \in [0, 1]$ that mixes the two stream logits position-by-position; a residual head h_R reads the concatenated stream pools to recover cross-stream signal.

$$\hat{y}(x) \;=\; \sigma\Big(\; \underbrace{\alpha(x)}_{\text{phase gate}} \cdot \underbrace{h_K(\phi_K(x))}_{\text{king stream}} + \big(1-\alpha(x)\big)\cdot \underbrace{h_E(\phi_E(x))}_{\text{exchange stream}} + \underbrace{h_R\big(\phi_K(x)\,\oplus\,\phi_E(x)\big)}_{\text{residual}} \;\Big)$$

ϕ_E, ϕ_K are the per-stream conv encoders; $\oplus$ denotes channel-wise concatenation; σ the binary sigmoid.

> *Hard-encodes the classical Stockfish-style decomposition (exchange evaluation + king safety) as architecture instead of asking the network to discover it from data.*

WHAT IT GIVES UP

Pure conv inside each stream means long-range piece interactions cost many layers. Mitigated at scale by replacing each stream's encoder with attention.

i048 — Rule-Automorphism Quotient Bottleneck Network (RAQ-Net)

simple_18 params **179k** speed **12.0k/s** test PR AUC **0.861**

Treats the chess board as a G-set under the discrete chess automorphism group $G = \langle \mu_{\mathrm{LR}}, \mu_{\mathrm{col}} \rangle$ (horizontal mirror with castling-rights swap, and colour-flip with role swap). The trunk is G-equivariant by construction; the classifier reads from the orbit space rather than the raw board.

$$\Phi(x) \;=\; \frac{1}{|G|}\sum_{g\in G} \rho(g)\cdot\phi\big(g^{-1}\cdot x\big) \qquad \text{with} \qquad \Phi\big(g\cdot x\big) = \rho(g)\,\Phi(x) \quad \forall g \in G$$

ϕ is the per-orbit encoder; $\rho(g)$ realigns equivariant outputs after the group action; the second equation is the equivariance constraint Φ satisfies by construction.

> *Symmetries that the rest of the field has to learn from data are baked into the architecture. Sample-efficiency advantage is mathematically guaranteed.*

WHAT IT GIVES UP

The quotient bottleneck discards information by construction; useful for binary discrimination, less so for value/policy regression.

i018 — Oriented Tactical Sheaf Laplacian

simple_18 params **91k** speed **5.5k/s** test PR AUC **0.861**

Builds a sheaf over chess squares whose sections encode *directed* tactical incidence (attacker → defender, side-to-move-aware). Convolves the input with the oriented sheaf Laplacian L_{or} instead of a standard graph Laplacian, preserving the chirality of attack.

$$L_{\text{or}} \;=\; D - A_{\text{or}}, \qquad A_{\text{or}}[u,v] \;=\; \begin{cases} +1 & u \text{ attacks } v \\ -1 & v \text{ attacks } u \\ 0 & \text{otherwise} \end{cases} \qquad y = W\, L_{\text{or}}\, x$$

D is the in/out-degree diagonal; A_{or} is signed by attacker/defender role and side-to-move; W is a learnable mixing matrix.

> *Tactical relations on a chess board are intrinsically directed; a standard symmetric graph Laplacian throws this away. The oriented Laplacian preserves it.*

WHAT IT GIVES UP

Tiny parameter budget (91k) makes the architecture itself a hyperparameter — needs tuning at scale before its true ceiling is known.

i188 — Tactical Program Induction Network

simple_18 params **710k** speed **8.3k/s** test PR AUC **0.861**

Treats a puzzle as the existence of a short tactical program (sacrifice → fork → promotion). A neural program-induction head scores candidate programs over a learned library $\mathcal{P}$ of tactical primitives. The puzzle logit is the maximum-likelihood program score.

$$p(\text{puzzle} \mid x) \;=\; \max_{P \in \mathcal{P}} q_\theta(P \mid x), \quad q_\theta(P \mid x) \;=\; \prod_{t=1}^{|P|} q_\theta\big(P_t \mid x, P_{<t}\big)$$

$\mathcal{P}$ is the (learned) library of short tactical programs; q_θ is an autoregressive neural program scorer.

> *Tactical solutions are* compositional sequences of primitives*, not point-evaluations; a program-shaped output head exposes that compositionality.*

WHAT IT GIVES UP

Program-induction heads are notoriously hard to train; high parameter count makes this less Pareto-efficient than the smaller winners.

BT4 — LC0 BT4 (reference architecture)

lc0_bt4_112 params ~**50M (medium)** speed **varies** test PR AUC **n/a (not in scout)**

BT4 is the current LC0 reference trunk: a piece-token transformer over the 64 squares of the board. The input is the 112-plane LC0 encoding (8 history steps × 13 piece-channel planes plus auxiliary planes for castling rights, side-to-move, en-passant, half-move clock). The trunk applies a stack of multi-head self-attention blocks where each square is a token; positional information is injected via per-square learned embeddings rather than 2D positional encoding. Two heads sit on top: a WDL value head and an 1858-dim move-policy head.

$$\text{BT4}(x) \;=\; \text{Head}\Big(\underbrace{\big(\text{Block}_L \circ \cdots \circ \text{Block}_1\big)}_{\text{stack of } L \text{ transformer blocks}} \big(\text{Embed}(x) + P\big) \Big), \quad \text{Block}_\ell(z) = \text{FFN}\big(\text{MHSA}(z)\big) + z$$

$\text{Embed} : \mathbb{R}^{112\times 8\times 8} \to \mathbb{R}^{64\times d}$ *projects each square's plane vector to a d-dim token; $P \in \mathbb{R}^{64\times d}$ is the learned per-square positional embedding;* MHSA *is multi-head self-attention over all* 64 *tokens;* FFN *is a 2-layer feed-forward.*

> *Square-as-token allows any square pair to interact in a single layer — the right prior for long-range piece relationships like queen-on-a1 attacking king-on-h8. This is the structural property other trunks struggle to match.*

WHAT IT GIVES UP

The trunk is *generic*: it has no chess-specific decomposition, no chess group equivariance, no chess-aware

attention bias. All of those have to be learned from data. The headline strength of LC0 BT4 comes from training-budget, not from the architecture itself.

i011 — VetoSelect Positive-Claim Abstention

lc0_bt4_112 params **502k** speed **11.7k/s** test PR AUC **0.858**

Adds a selective-abstention head on top of an LC0 BT4-style trunk. The loss separates raw positive evidence from refuting evidence, and applies an explicit penalty on hard near-puzzle negatives (the "veto").

$$\mathcal{L} \;=\; \underbrace{-\,y \log p^{+} - (1-y)\log(1-p^{+})}_{\text{binary cross-entropy}} \;+\; \lambda\, \mathbb{E}_{x \,\in\, \text{hard}^{-}}\big[\, p^{+}(x) \,\big]$$

p^{+} is the puzzle logit; hard^{-} *is the set of verified-near-puzzle negatives (fine label 1);* $\lambda > 0$ *is the veto weight.*

> *Aggregate PR AUC is the wrong scoreboard for puzzle classification — the real cost is false positives on positions that look tactical but aren't. Explicit suppression of that mode is a strict gain.*

WHAT IT GIVES UP

Multi-objective loss trades aggregate AUC capacity for slice robustness. Useful if matched-recall FP on near-puzzles is what you care about; less useful if you only quote aggregate AUC.